
Contents

1	Introduction	3
2	Related Work	3
2.1	Handwritten Digit Recognition and MNIST	3
2.2	Arabic Handwritten Character Recognition	3
2.3	Federated Learning for Handwriting Recognition	3
3	Datasets for Arabic Handwritten Character Recognition	3
3.1	CNN Architecture on OIHACBD-28 Dataset	4
3.1.1	Randomly Initialized Weights	4
3.1.2	Scheduled Randomized Weights	4
3.1.3	Data Augmentation	5
3.1.4	Resizing to 32×32	6
3.1.5	Comparing	7
3.2	HMBD Dataset v1	7
3.2.1	Preprocessing	7
3.2.2	Training the Model on the New Dataset	8
3.3	AHCD Dataset	9
4	Centralized Learning	10
4.1	CNN Architecture	10
4.1.1	Results	10
4.2	More complex CNN architecture	10
4.3	CNN Architecture for Arabic Handwritten Letters	11
4.3.1	Results	11
4.4	ResNet-18	12
4.4.1	Results	12
4.5	Hybrid Attention CNN	13
4.6	Comparative Analysis	13
5	Federated Learning	13
5.1	Flower Federated Learning	14
5.2	Flower Federated Learning with Missing Classes	14
5.3	Results	15
5.3.1	Classes with Low Recognition Performance	15
5.4	Flower Federated Learning with More Balanced Missing Classes	16
5.4.1	Results	16
5.5	Comparative Study: Centralized vs Federated Learning	16
6	Conclusion	16

List of Figures

1	Hardware specifications of the machine used in this project.	3
2	Training metrics using random weight initialization.	4
3	Learning rate schedule: Linear warm-up followed by cosine cool-down [4].	5
4	Training metrics with scheduled learning rate application.	5
5	Confusion Matrix	6
6	Training and validation metrics with data augmentation.	6
7	Preprocessing of images: comparison of different resizing and localization strategies.	7
8	Different writing positions of the letter <i>Ain</i> in Arabic script	8
9	Training and validation metrics on the new Arabic dataset. Validation accuracy reached 96.40%, showing stable convergence and good generalization.	9
10	Training and validation metrics for the AHCD dataset, showing loss function and validation accuracy over epochs.	9
11	Per-class accuracy for the AHCD dataset. The lowest accuracy corresponds to the letter Zatl.	9
12	Performance of the CNN on the centralized dataset.	10
13	Accuracy of CNN	11
14	Training metrics of the ResNet-18 model on our dataset.	12
15	Datasets represented as independent clients	14
16	Training metrics (left) and per-class accuracy (right) during federated learning.	14
17	Final performance of the federated learning model.	14
18	Detected character classes after federated training with missing classes at clients	15
19	Training and validation metrics of the federated model with missing classes	15
20	Per-letter performance analysis highlighting low-frequency and difficult classes	15
21	Per-letter performance after balancing class distribution across clients	16

1 Introduction

Handwritten character recognition is a fundamental problem in pattern recognition and computer vision, with applications in document digitization, postal address reading, and bank check processing. While handwritten digit recognition achieves near-human performance on MNIST, Arabic handwritten character recognition remains challenging due to the cursive nature of the script, multiple contextual forms (isolated, beginning, middle, end), and letters differing only by diacritical dots. Variations in handwriting style, stroke thickness, scale, and orientation further increase complexity.

Deep learning, particularly Convolutional Neural Networks (CNNs), has significantly improved performance. Classic architectures like LeNet-5, initially developed for MNIST, introduced key concepts such as local receptive fields, weight sharing, and subsampling. More recent models, including deeper CNNs and residual networks (ResNet), have further enhanced recognition capabilities. However, these approaches usually require centralized training, where all data is collected in a single location.

Centralized learning has limitations when data is distributed across devices or institutions due to privacy concerns. Federated Learning (FL) addresses this by allowing clients to train local models on private data and share only model updates with a central server. This is particularly suitable for handwriting recognition, where data is naturally decentralized and heterogeneous.

In this work, we investigate Arabic handwritten character recognition under both centralized and federated learning. Experiments are conducted on three benchmark datasets—AHCD, HMBD, and OIHACDB—treated as independent clients. We evaluate multiple CNN architectures under centralized training, including a baseline CNN, an improved CNN, ResNet-18, and a hybrid attention model. A lightweight baseline CNN is selected for federated learning experiments. We also analyze the impact of data heterogeneity and scenarios where each letter exists in only one dataset.

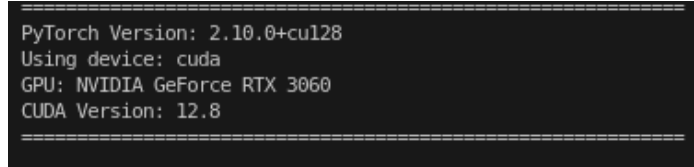


Figure 1: Hardware specifications of the machine used in this project.

2 Related Work

2.1 Handwritten Digit Recognition and MNIST

Handwritten digit recognition, particularly using the MNIST dataset, has long served as a benchmark for evaluating machine learning models. MNIST consists of grayscale images of handwritten digits and has been widely used to validate new architectures and optimization techniques. LeNet-5 was one of the first CNNs to achieve strong performance, demonstrating hierarchical feature learning from raw pixels. Subsequent works explored deeper CNNs and residual networks, which remain reference architectures for evaluating generalization, training stability, and distributed learning algorithms such as federated learning.

2.2 Arabic Handwritten Character Recognition

Arabic character recognition is more challenging due to script complexity. Datasets like AHCD, HMBD, ADAB, and OIHACDB have facilitated research. Early approaches relied on handcrafted features with classical classifiers such as SVMs or HMMs. Deep learning, especially CNN-based methods, has become dominant, achieving high accuracy through data augmentation and preprocessing to handle letter similarity and writing style variations. Most prior work assumes centralized access to all training data.

2.3 Federated Learning for Handwriting Recognition

Federated Learning (FL) enables collaborative training while preserving data privacy. The Federated Averaging (FedAvg) algorithm is widely used for aggregating client models. MNIST is a common benchmark for FL under non-IID conditions. Handwriting recognition is particularly sensitive to heterogeneous client data, as different users may have distinct styles or class distributions. Few studies address FL for Arabic handwritten character recognition, motivating our work to analyze centralized vs. federated training using multiple Arabic handwriting datasets.

3 Datasets for Arabic Handwritten Character Recognition

In this work, we utilize three benchmark datasets for evaluating Arabic handwritten character recognition methods.

1. **Arabic Handwritten Characters Dataset (AHCD)**: Developed by Loey et al. [1], AHCD contains 16,800 characters written by 60 participants, standardized to 32×32 pixels. The dataset is partitioned into training

(13,440 characters, 480 per class) and test sets (3,360 characters, 120 per class). AHCD has been widely used in CNN-based Arabic character recognition experiments. The dataset is [available on Kaggle](#).

2. **HMBD Dataset v1:** Developed by Hossam Balaha et al. [2], HMBD v1 contains 54,115 images across 115 classes, collected from 125 volunteers. The dataset captures Arabic characters in all four positions (isolated, beginning, middle, end) as well as numbers. All images are 300×300 pixels with white background and black characters. HMBD v1 provides extensive variability for handwriting recognition research and is [available on GitHub](#).
3. **OIHACDB-28 Dataset:** The Off-line Isolated Handwritten Arabic Characters Database (OIHACDB-28) was developed by Boufenar et al. [3] to provide a benchmark for CNN-based Arabic character recognition. It contains 5,600 images of isolated Arabic letters, with 28 classes and exactly 200 samples per class. Images are standardized to 128×128 pixels in BMP format, making the dataset suitable for offline handwritten Arabic character recognition experiments. The dataset is [available via MediaFire](#).

Dataset	Image Size	Number of Images	Number of Classes
AHCD	32×32	16,800	28
HMBD v1	300×300	54,115	28 letters $\times$ 4 positions
OIHACDB-28	128×128	5600	28

Table 1: Comparison of the three Arabic handwritten character datasets

then when i get best accuracy i try centralied model from scratch by me and famous architecture
and then do fedrated learning and test
and then do fedrated learning each letters exist only in one data sets

3.1 CNN Architecture on OIHACBD-28 Dataset

To evaluate the OIHACBD-28 dataset, we designed a Convolutional Neural Network (CNN) consisting of three feature extraction blocks followed by a dense classifier. The model processes inputs of size $128 \times 128 \times 3$. The architecture is detailed below:

- **Block 1:** Conv($3 \rightarrow 32$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $64 \times 64 \times 32$
- **Block 2:** Conv($32 \rightarrow 64$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $32 \times 32 \times 64$
- **Block 3:** Conv($64 \rightarrow 128$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $16 \times 16 \times 128$
- **Classifier:** Flatten $\rightarrow$ Linear($32768 \rightarrow 512$) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ Linear($512 \rightarrow \text{num_classes}$)

3.1.1 Randomly Initialized Weights

In the first phase of our experiment, we tested the architecture using standard random weight initialization.

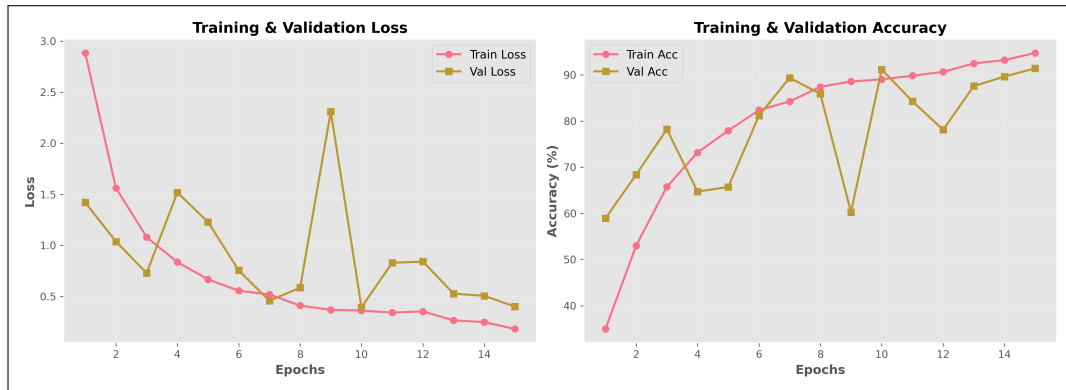


Figure 2: Training metrics using random weight initialization.

From the results in Figure 2, we observe that **the validation plot was initially unstable (volatile) but started to stabilize after epoch 13**, achieving an **average accuracy of 91%**. The training time was approximately 47 seconds total.

3.1.2 Scheduled Randomized Weights

To optimize convergence, we implemented a learning rate schedule featuring a linear warm-up followed by a cosine cool-down, as proposed by Kevin Murphy in *Probabilistic Machine Learning* [4].

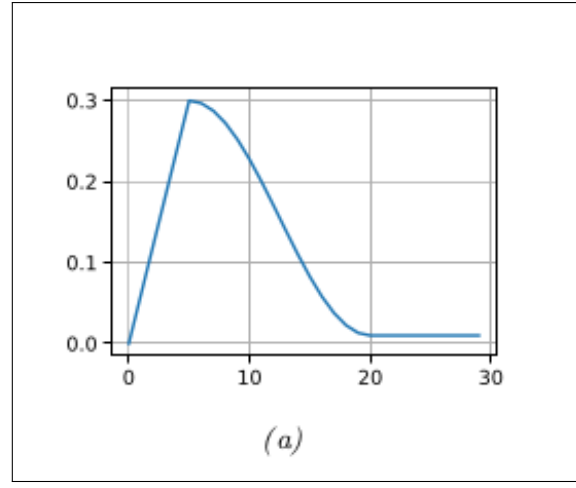


Figure 3: Learning rate schedule: Linear warm-up followed by cosine cool-down [4].

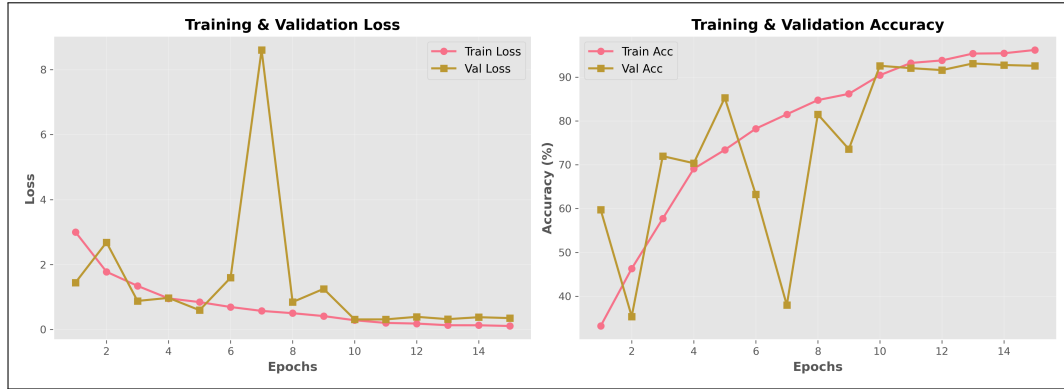


Figure 4: Training metrics with scheduled learning rate application.

By applying this schedule, we observed a significant improvement in training dynamics. As seen in Figure 4, the model achieved **better and faster convergence with 93% accuracy**, and the accuracy **stabilized after only 10 epochs**.

Analysis: While both methods required roughly 47 seconds total, the scheduled approach effectively reduces total training time by reaching a stable state three epochs earlier. This indicates that a dynamic learning rate helps the model navigate the loss landscape more efficiently than a static initialization approach.

Observation

From the Confusion Matrix Above , the model struggles the most with the following letters:

- **Dall** (recall = 0.8444) – often confused with **Raal** due to visual similarity.
- **SiinI** (recall = 0.8750) – sometimes confused with **Sadl**
- **ThaI** (recall = 0.8636) – occasionally confused with **Nounl,Taal**,

This indicates that the model has difficulty distinguishing letters with similar shapes, while performing very well on most other classes.

3.1.3 Data Augmentation

Since the model struggles to distinguish similar letters, we applied data augmentation to improve its generalization. The augmentation includes various transformations such as rotations, scaling, translations, and shearing.

To avoid data leakage, augmentation is applied **only after selecting the training samples**. If we augmented the entire dataset before splitting, some augmented versions of the same images could appear in both the training and validation sets, causing obvious data leakage. By generating new augmented samples at every epoch **only for the training set**, the model is exposed to diverse examples, improving its ability to generalize.

With these augmentations, the model achieved an accuracy of **98.3%** on the validation set.

As shown in Figure 6, the validation accuracy is slightly higher than the training accuracy. This is reasonable because the training dataset is augmented with more challenging variations, whereas the validation set remains clean and unaltered.

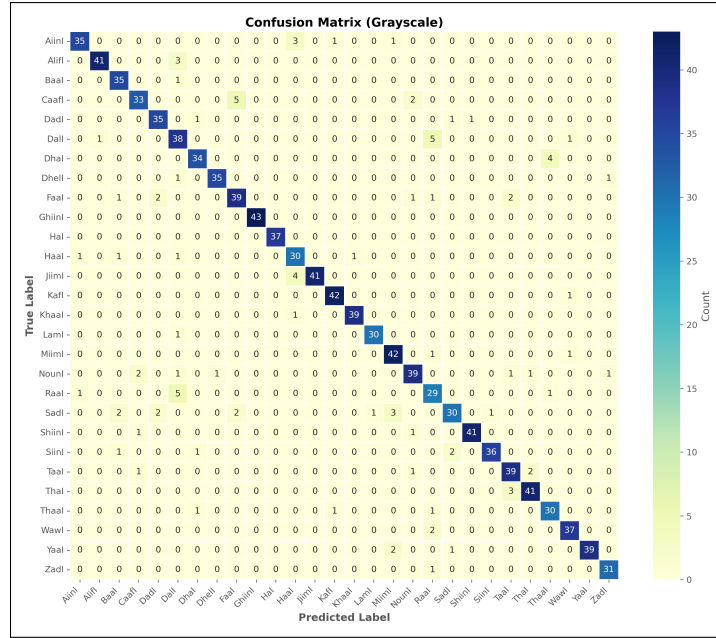


Figure 5: Confusion Matrix

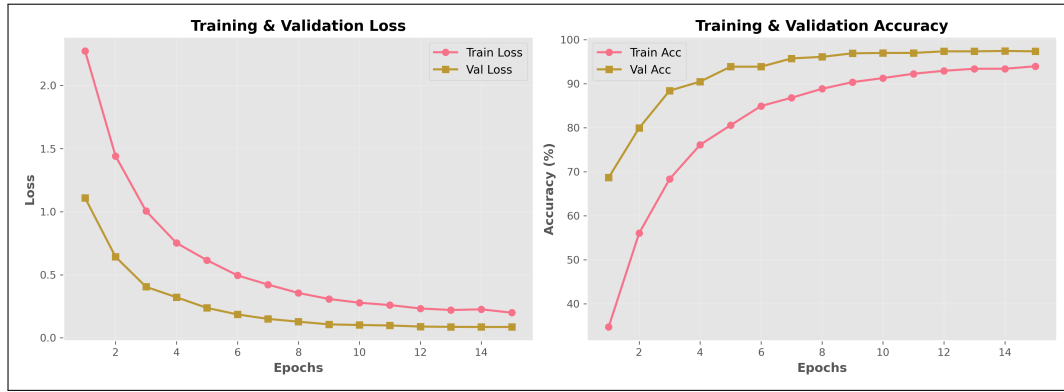


Figure 6: Training and validation metrics with data augmentation.

Overall, both training and validation metrics are steadily improving, indicating that the model is learning effectively and generalizing well without overfitting.

Summary of Improvements:

- Initial architecture achieved **91%** accuracy with standard initialization.
- By optimizing weight initialization and training settings, accuracy increased to **93%**.
- Adding data augmentation to help the model generalize further boosted the accuracy to **97.68%**.

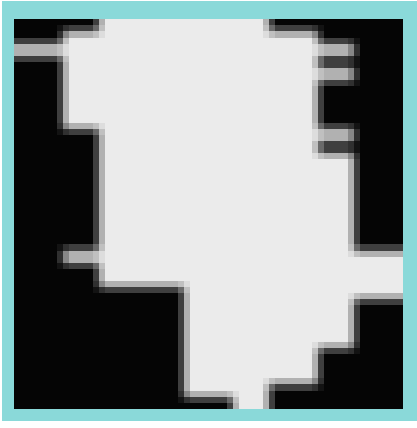
3.1.4 Resizing to 32×32

Table 2: CNN Architecture for 32×32 Grayscale Images (Arabic Letters)

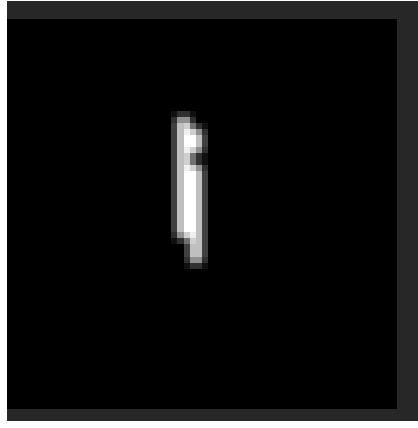
Block / Layer	Layer Details	Output Size
Block 1	Conv2d(1 → 32, 3×3, padding=1) → ReLU → BatchNorm2d(32) → MaxPool2d(2×2)	16×16×32
Block 2	Conv2d(32 → 64, 3×3, padding=1) → ReLU → BatchNorm2d(64) → MaxPool2d(2×2)	8×8×64
Block 3	Conv2d(64 → 128, 3×3, padding=1) → ReLU → BatchNorm2d(128) → MaxPool2d(2×2)	4×4×128
Classifier	Flatten → Linear(2048 → 512) → ReLU → Dropout(0.5) → Linear(512 → C)	1×C
Parameters		1,156,572

In The preprocessing part We encountered two problems localised then resized distorted the image resized only loses informations so i figured a solution is to localized to the smallest possible square then resize so we can keep the original image structure. For this experiment, we used a **localized and resized dataset**, where each character is **centered** within a 32×32 image. By reducing the input size, we were able to significantly simplify the model architecture.

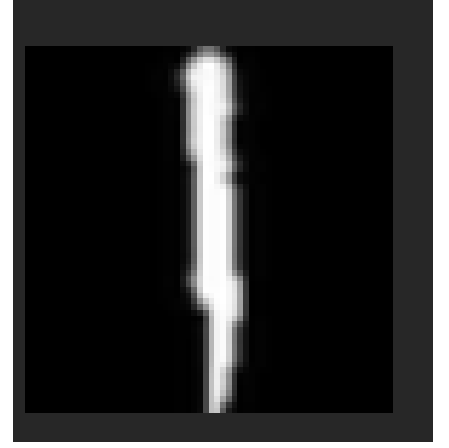
The total number of trainable parameters was reduced from **16,885,212** to **1,156,572**, which corresponds to a **93% decrease in parameters**. This drastic reduction not only lowers the memory requirements but also allows for much



(a) Localized then resized



(b) Resized only



(c) Localized to smallest square then resized

Figure 7: Preprocessing of images: comparison of different resizing and localization strategies.

faster training.

Indeed, the **training time decreased from approximately 47 seconds to 13.66 seconds**, while maintaining a high validation accuracy of **97.86%** (slightly higher than the previous 97.68%). This demonstrates that the model achieves **faster convergence with far fewer parameters**, without sacrificing performance.

In summary, resizing to 32×32 results in a **more efficient model**, achieving almost the same (slightly better) accuracy in roughly **one-third of the training time**. This is a significant improvement in both **computational efficiency** and **training speed**, making the model more practical for real-world applications.

3.1.5 Comparing

Table 3: Comparison of different techniques and their performance on the model

Technique	Description	Accuracy (%)	Training Time (s)
Randomly Initialized Weights	Standard random initialization	91.0	47
Scheduled Randomized Weights	Learning rate schedule	93.0	47
Data Augmentation	Rotations, scaling, translations, shearing	97.68	47
Resizing to 32×32	93% decrease in parameters	97.86	13.66

3.2 HMBD Dataset v1

We Will conduct expirement like we did with **OIHACBD-28** so data **augmentation and resizing (localised then resized)**, but there one step which also include is inverting the colors of these images and removing other positions of letters so it more unified with other datasets for cleaner **FL**.

After this step, we will keep only the **isolated form** of each letter and number and remove the other three positions (start, middle, end). This allows us to match our dataset format with other datasets and ensures fair and consistent learning in FL.

3.2.1 Preprocessing

To ensure consistent labeling and compatibility with the OIHACBD dataset, we prepared a new dataset following a series of preprocessing steps. Each step is explained below, along with the corresponding Python scripts used.

1. **Combine all images into a single folder:** Using `one_folder.py`, we output all images into a single folder named `isolated_only`.
 - **Purpose:** To unify the dataset structure, making it easier to process and label images consistently for training.
2. **Invert image colors:** Using `inverted_cropped.py`, we inverted the colors so that the letters are represented in white (value 1 in grayscale) and the background is black. Additionally, we localized the letters to the smallest possible square.
 - **Purpose:** This improves contrast and ensures that each image contains only the letter, eliminating unnecessary background pixels.
3. **Resize images:** Using `resize.py`, we resized all images to 32×32 pixels.
 - **Purpose:** Standardizes image dimensions for input into the CNN model, reducing computational cost and ensuring consistent input size.

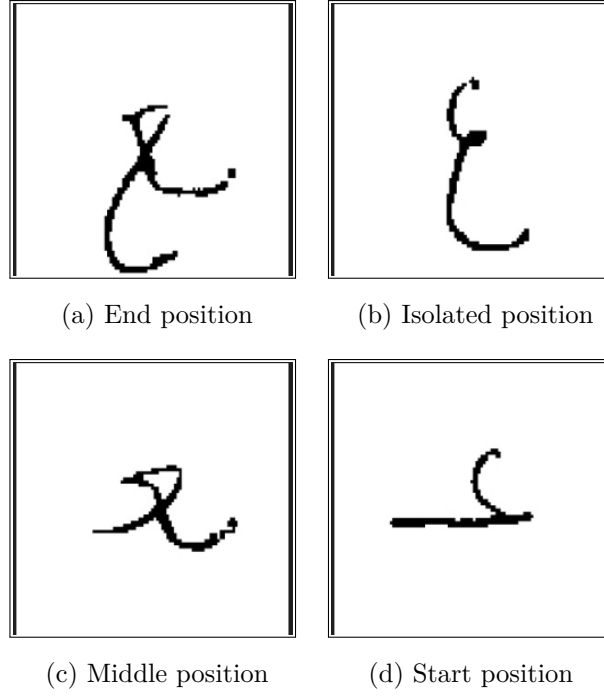


Figure 8: Different writing positions of the letter *Ain* in Arabic script

4. **Adjust image names:** Using `adjustment.py`, we renamed all images to match the OIHACBD labeling scheme.
 - **Purpose:** Ensures that the new dataset has consistent class names and file naming conventions, which is crucial for training and evaluation.

Names

After performing the preprocessing steps above, the number of images per class is as follows:

- | | | | |
|----------------------|----------------------|----------------------|---------------------|
| • AiinI: 462 | • FaaI: 464 | • LamI: 1852 | • ThaI: 460 |
| • AlifI: 1430 | • GhiinI: 920 | • MiimI: 468 | • ThaaI: 457 |
| • BaaI: 468 | • HaI: 468 | • NounI: 466 | • WawI: 916 |
| • CaafI: 467 | • HaaI: 477 | • RaaI: 476 | • YaaI: 924 |
| • DadI: 455 | • HamzaI: 466 | • SadI: 459 | • ZadI: 479 |
| • DallI: 472 | • JiimI: 472 | • ShiinI: 460 | |
| • DhaI: 474 | • KafI: 464 | • SiinI: 468 | |
| • DhellI: 458 | • KhaaI: 476 | • TaaI: 467 | |

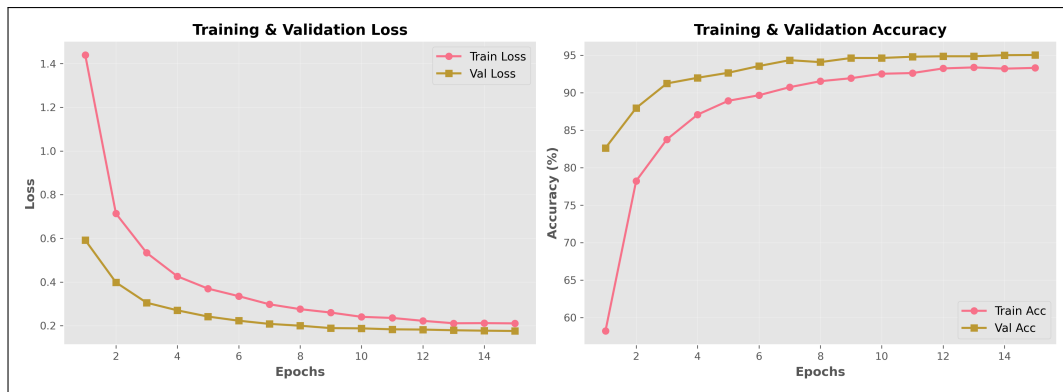
Total number of images: 17,495 this is the only dataset that will shrink after preprocessing since it has position of letters deleted.

3.2.2 Training the Model on the New Dataset

We trained the CNN using the same architecture as before, this time on the new dataset prepared with **32×32 localized and inverted images** and **data augmentation**. The model achieved an accuracy of **96.40%** on the validation set and took about **45s**.

Although this accuracy is slightly lower than the previous experiment with the OIHACBD dataset, it still represents excellent performance and demonstrates the model's ability to generalize to new, slightly different data.

As shown in Figure 9, both training and validation metrics steadily improve over epochs. The slight difference between training and validation accuracy is expected because the training set is augmented with various transformations, while the validation set remains clean and unaltered.



Observation

- The model generalizes well to the new dataset, achieving **96.40% accuracy**.
- The small drop in accuracy compared to the original dataset (e.g., 97.68%) is likely due to **differences in the dataset distribution and class imbalance**.
- Using **data augmentation** continues to be essential for improving generalization on letters that are visually similar.
- the model still convergence faster around 10 epoches but we are keeping 15 for experimenting

3.3 AHCD Dataset

The **AHCD dataset** is organized similarly to the OHIACBD dataset. This dataset already contains images of size 32×32 , so our main preprocessing step was to unify the file names and merge all images into a single folder. After this step, the dataset was ready for training.

We trained a **Convolutional Neural Network (CNN)** using the same architecture as before. The model achieved an **overall accuracy of 96.52%** (3243/3360) after **45 seconds of training**.

The training process, including both the training loss and the validation loss, as well as the validation accuracy, is illustrated in Figure 10.

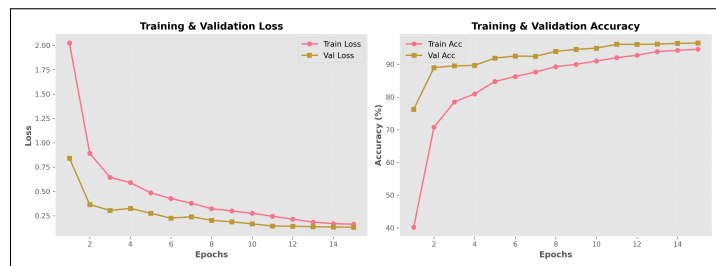


Figure 10: Training and validation metrics for the AHCD dataset, showing loss function and validation accuracy over epochs.

We also analyzed the **accuracy per class** for each letter. As shown in Figure 11, the model performed very well on most letters. The lowest accuracy was **89.9%** for the letter **Zadl**.

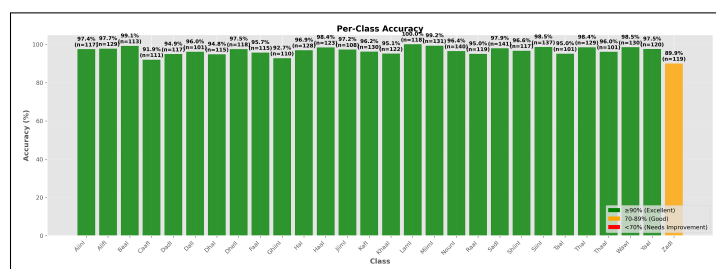


Figure 11: Per-class accuracy for the AHCD dataset. The lowest accuracy corresponds to the letter Zatl.

4 Centralized Learning

In this experiment, we combine all available datasets into a single centralized dataset and train the model on it. The final dataset contains a total of 39,645 images.

4.1 CNN Architecture

We use the same CNN architecture as in the previous experiments:

- **Block 1:** Conv($3 \rightarrow 32$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $64 \times 64 \times 32$
- **Block 2:** Conv($32 \rightarrow 64$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $32 \times 32 \times 64$
- **Block 3:** Conv($64 \rightarrow 128$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $16 \times 16 \times 128$
- **Classifier:** Flatten $\rightarrow$ Linear($32768 \rightarrow 512$) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ Linear($512 \rightarrow \text{num_classes}$)

4.1.1 Results

The model achieved an overall accuracy of **96.50%** (5738/5946) with a total of **208 misclassified samples** and took **170.11 seconds** to train with **1,157,085 parameters**. The training and evaluation curves are shown in Figure 12.

Despite the larger dataset, the accuracy remains similar to previous experiments. This suggests that the model is **underfitting**, meaning that the architecture is too simple to fully exploit the additional data. We therefore try a more complex model in the next experiment.

In addition, the accuracy distribution across letter classes is more uniform compared to previous datasets. This indicates better generalization of the model when trained on the centralized dataset.

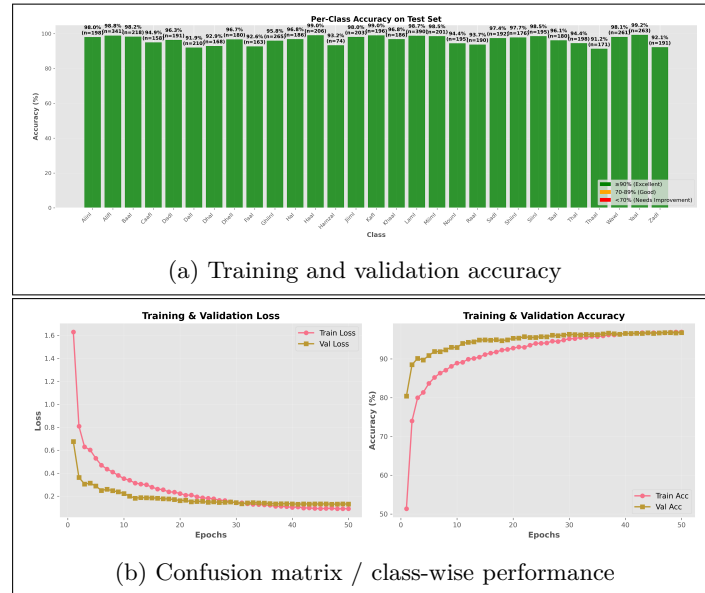


Figure 12: Performance of the CNN on the centralized dataset.

4.2 More complex CNN architecture

- **Block 1:** Conv($1 \rightarrow 32$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Conv($32 \rightarrow 32$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $64 \times 64 \times 32$
- **Block 2:** Conv($32 \rightarrow 64$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Conv($64 \rightarrow 64$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $32 \times 32 \times 64$
- **Block 3:** Conv($64 \rightarrow 128$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Conv($128 \rightarrow 128$, 3×3) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ AdaptiveAvgPool(1×1) $\rightarrow$ Output: $1 \times 1 \times 128$
- **Classifier:** Flatten $\rightarrow$ Linear($128 \rightarrow 256$) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ Linear($256 \rightarrow 28$)

Additions and Modifications

Compared to a standard CNN architecture, we added the following enhancements:

- **Double convolution per block:** Each block has two convolution layers with BatchNorm and ReLU for better feature extraction.
- **Batch Normalization:** Added after every convolution layer to stabilize and accelerate training.
- **Adaptive Average Pooling:** Reduces the feature map to 1×1 , making the classifier input independent of the input image size.

- **Dropout:** Added in the classifier to reduce overfitting.

4.3 CNN Architecture for Arabic Handwritten Letters

The proposed CNN model is designed to recognize Arabic handwritten characters. Its architecture is as follows:

- **Block 1:** Conv(1 → 32, 3×3) → ReLU → BatchNorm → Conv(32 → 32, 3×3) → ReLU → BatchNorm → MaxPool(2×2) → Output: $64 \times 64 \times 32$
- **Block 2:** Conv(32 → 64, 3×3) → ReLU → BatchNorm → Conv(64 → 64, 3×3) → ReLU → BatchNorm → MaxPool(2×2) → Output: $32 \times 32 \times 64$
- **Block 3:** Conv(64 → 128, 3×3) → ReLU → BatchNorm → Conv(128 → 128, 3×3) → ReLU → BatchNorm → AdaptiveAvgPool(1×1) → Output: $1 \times 1 \times 128$
- **Classifier:** Flatten → Linear(128 → 256) → ReLU → Dropout(0.5) → Linear(256 → 28)

Additions and Modifications

Compared to a standard CNN, the following enhancements were added:

- **Double Convolutions per Block:** Each block contains two convolutional layers for better feature extraction.
- **Batch Normalization:** Added after every convolutional layer to stabilize and accelerate training.
- **Adaptive Average Pooling:** Reduces the feature map to 1×1 , making the classifier input independent of the input image size.
- **Dropout:** Added in the classifier to reduce overfitting.

4.3.1 Results

The proposed CNN model achieved a **test accuracy of 97.19%**. The training process was completed in approximately **7 minutes and 37 seconds**.

The network contains a total of **316,652 parameters**, distributed across convolutional and fully connected layers as follows:

- **Block 1:** Conv1 ($1 \rightarrow 32, 3 \times 3$) = 320 parameters, Conv2 ($32 \rightarrow 32, 3 \times 3$) = 9,248 parameters
- **Block 2:** Conv1 ($32 \rightarrow 64, 3 \times 3$) = 18,496 parameters, Conv2 ($64 \rightarrow 64, 3 \times 3$) = 36,928 parameters
- **Block 3:** Conv1 ($64 \rightarrow 128, 3 \times 3$) = 73,856 parameters, Conv2 ($128 \rightarrow 128, 3 \times 3$) = 147,584 parameters
- **Classifier:** Linear($128 \rightarrow 256$) = 33,024 parameters, Linear($256 \rightarrow 28$) = 7,196 parameters

These results indicate that the network is both **accurate** and **efficient**, achieving high performance with a relatively low number of parameters.

Epoch 42	Train Acc 97.24%	Val Acc 96.75%
Epoch 43	Train Acc 97.29%	Val Acc 97.02%
Epoch 44	Train Acc 97.37%	Val Acc 96.92%
Epoch 45	Train Acc 97.43%	Val Acc 96.97%
Epoch 46	Train Acc 97.24%	Val Acc 97.02%
Epoch 47	Train Acc 97.38%	Val Acc 97.04%
Epoch 48	Train Acc 97.50%	Val Acc 97.12%
Epoch 49	Train Acc 97.60%	Val Acc 97.02%
Epoch 50	Train Acc 97.55%	Val Acc 97.19%

Figure 13: Accuracy of CNN

4.4 ResNet-18

ResNet-18 is a well-known convolutional neural network architecture that utilizes residual connections to ease the training of deep networks. For small grayscale images of size 32×32 , we adapted the standard ResNet-18 by removing the initial max-pooling layer to preserve spatial resolution.

Table 4: ResNet-18 Architecture for 32×32 Grayscale Images

Stage / Block	Layer Type	Output Size	Notes / Parameters
Input	Image	$32 \times 32 \times 1$	Grayscale image
Conv1	Conv 3×3 , 64, stride 1 BatchNorm + ReLU	$32 \times 32 \times 64$ $32 \times 32 \times 64$	No initial maxpool (small image) -
Layer 1	BasicBlock $\times 2$ -	$32 \times 32 \times 64$ -	Residual connection, stride=1 3×3 conv, BN, ReLU
Layer 2	BasicBlock $\times 2$ -	$16 \times 16 \times 128$ -	Downsample first block, stride=2 Residual connection with 1×1 conv
Layer 3	BasicBlock $\times 2$ -	$8 \times 8 \times 256$ -	Downsample first block, stride=2 Residual connection with 1×1 conv
Layer 4	BasicBlock $\times 2$ -	$4 \times 4 \times 512$ -	Downsample first block, stride=2 Residual connection with 1×1 conv
Pooling	Adaptive AvgPool 1×1	$1 \times 1 \times 512$	Global average pooling
Classifier	Flatten + Linear	$1 \times 1 \times \text{num_classes}$	Fully connected layer, output=28 letters

4.4.1 Results

Our ResNet-18 model achieved an accuracy of 97.29% on the test set, with a total of 11,182,044 trainable parameters. Training took approximately 1282.9 seconds (20 minutes).

Although this configuration already yields high accuracy, experimenting with larger or deeper models could potentially improve performance. However, such models typically require significantly more parameters and longer training times.

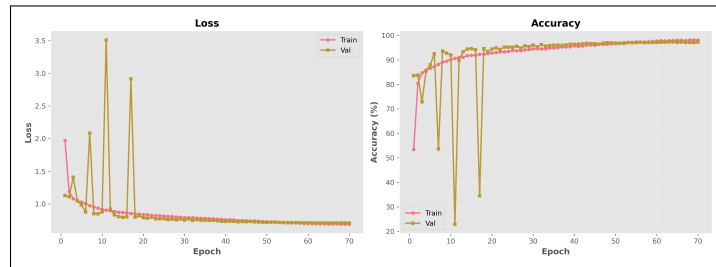


Figure 14: Training metrics of the ResNet-18 model on our dataset.

4.5 Hybrid Attention CNN

Table 5 presents the detailed architecture of the **SpatialPreservingArabicCNN**, a hybrid attention-based model designed for Arabic handwritten character recognition. The network integrates multi-scale feature extraction, channel and spatial attention (CBAM), as well as SE- and Coord-Attention mechanisms to enhance feature representation.

Table 5: Detailed Architecture of SpatialPreservingArabicCNN

Stage	Module	Output Dim	Configuration / Hyperparameters
Input	-	$32 \times 32 \times 1$	Grayscale Arabic Letter Image
Stem	Conv Block	$32 \times 32 \times 64$	3×3 Conv, Stride 1, BN, ReLU
Multi-Scale Context (MSC)	Branch 1	$32 \times 32 \times 32$	1×1 Conv (Local features)
	Branch 2	$32 \times 32 \times 32$	3×3 Dilated Conv (rate=2)
	Branch 3	$32 \times 32 \times 32$	3×3 Dilated Conv (rate=4)
	Branch 4	$32 \times 32 \times 32$	Global Avg Pool $\rightarrow 1 \times 1$ Conv $\rightarrow$ Upsample
	Fusion	$32 \times 32 \times 128$	Concat $\rightarrow 1 \times 1$ Conv $\rightarrow$ CBAM (Channel & Spatial)
Dense Attention Blocks	Block 1	$32 \times 32 \times 160$	DW-Conv (32 out), SE-Attention, Coord-Attention
	Block 2	$32 \times 32 \times 192$	DW-Conv (32 out), SE-Attention, Coord-Attention
	Block 3	$32 \times 32 \times 224$	DW-Conv (32 out), SE-Attention, Coord-Attention
Final Conv	Downsampling	$8 \times 8 \times 256$	Two 3×3 Conv (Stride 2), BN, ReLU
Pooling	Att-Weighted	$1 \times 1 \times 256$	Conv-based attention mask weighting
Classifier	Fully Conn.	29	$256 \rightarrow 128 \rightarrow$ Dropout (0.4) $\rightarrow$ Softmax

Total Parameters: 1,244,560

Despite the high complexity of this model, it achieved a ****test accuracy of 95.75%**** on the Arabic handwritten letter dataset. The training process took approximately **40 minutes**, which is significantly longer than simpler CNN architectures, highlighting that such an elaborate attention-based network may be ****over-parameterized** for a relatively simple classification task.

4.6 Comparative Analysis

As shown by the comparative results, **all evaluated models achieve high classification accuracy** on the Arabic handwritten letter dataset. However, some architectures—most notably the **Hybrid Attention CNN**—are **overly complex for a relatively simple classification task**. Despite incorporating advanced attention mechanisms, this model does not yield a proportional improvement in accuracy and requires **significantly longer training time**, making it computationally inefficient.

In contrast, the **Baseline CNN model** demonstrates an **excellent trade-off between accuracy, model complexity, and training efficiency**. It achieves competitive performance while remaining lightweight and fast to train. For these reasons, the **Baseline CNN is selected as the reference model** in this work.

Consequently, **our federated learning study is conducted using the Baseline CNN architecture**, and all federated learning results will be **directly compared against its centralized training performance**. This choice enables a fair and meaningful evaluation of the impact of federated learning on both accuracy and convergence behavior.

Table 6: Comparison of Centralized Learning Models for Arabic Handwritten Letter Recognition

Model	Trainable Parameters	Training Time	Test Accuracy
Baseline CNN (Centralized)	1,157,085	170.11 s ($\approx$ 2.8 min)	96.50%
Improved CNN (Double Conv + AAP)	316,652	7 min 37 s	97.19%
ResNet-18 (Adapted for 32×32)	11,182,044	1,282.9 s ($\approx$ 21.4 min)	97.29%
Hybrid Attention CNN	1,244,560	$\approx$ 40 min	95.75%

5 Federated Learning

Each dataset is treated as an independent client. The AHCD, HMBD, and OIHACDB datasets remain locally stored at their respective clients. Each client trains a local model and sends only the learned parameters to the central server. The server aggregates these parameters to construct a global model, ensuring data privacy.

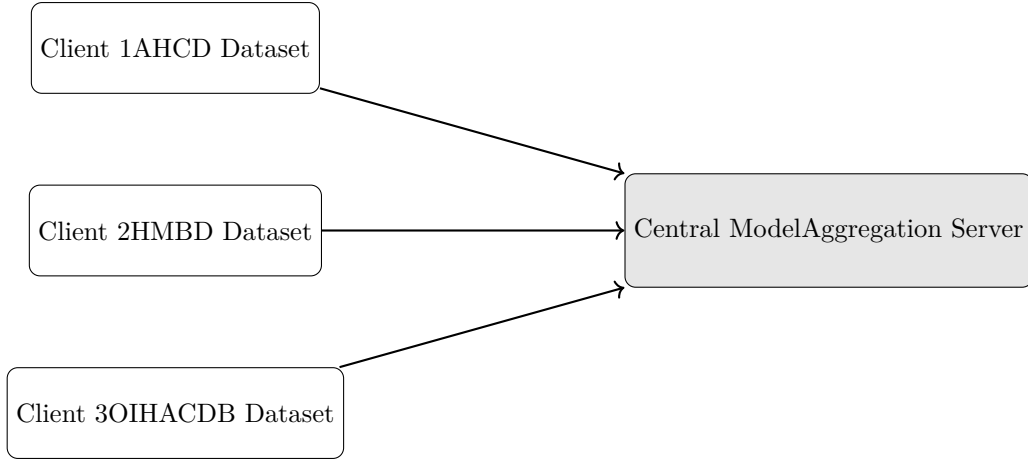


Figure 15: Datasets represented as independent clients

5.1 Flower Federated Learning

In this part, CUDA version 12.4 is required, We perform federated learning with three clients.

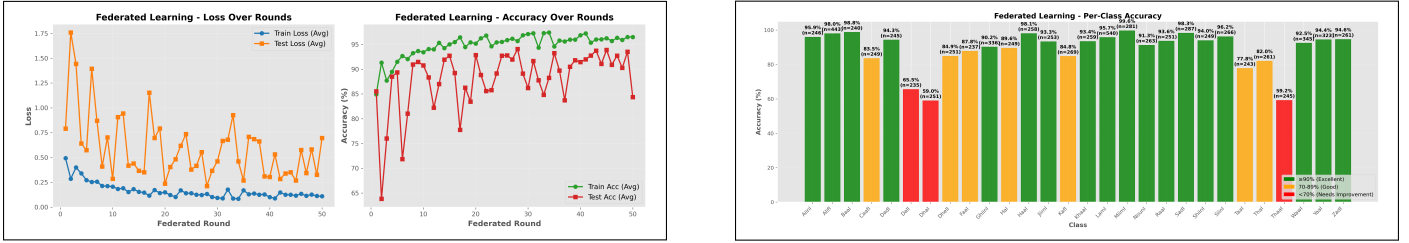


Figure 16: Training metrics (left) and per-class accuracy (right) during federated learning.

Overall Model Performance

- **Overall Accuracy (all data):** 91.50%
- **Total training time:** 711.32 seconds
- Letter Performance Distribution:**
 - **Excellent ($\geq 90\%$):** 20/28 letters (71.4%)
 - **Good (70–89%):** 6/28 letters (21.4%)
 - **Needs Work ($< 70\%$):** 2/28 letters (7.1%)

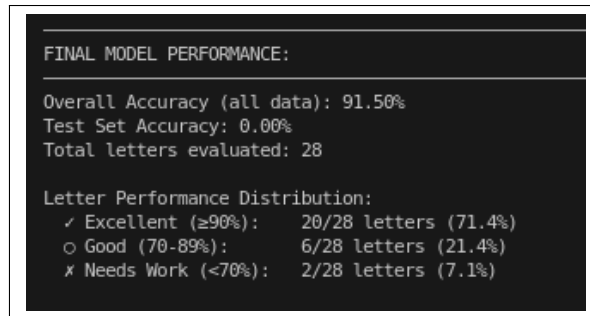


Figure 17: Final performance of the federated learning model.

5.2 Flower Federated Learning with Missing Classes

In this part, we simulate a federated learning scenario where the global model must learn even when some data are missing at each client. To do this, we provide each client with a unique subset of character classes (i.e., no client has access to the full set of Arabic letters). The global model is then evaluated on the complete set of classes. As expected, the model performance degrades due to the reduced and fragmented data distribution across clients. However, this experiment allows us to analyze how federated learning is affected by missing classes at the client level and to what extent the global model can generalize to unseen classes.

Client 1 (OIHCBD Dataset)

AiinI, AlifI, BaaI, CaafI, DadI,
DalI, DhaI, DheI, FaaI, GhiinI

Client 2 (HMBD Dataset)

HaI, HaaI, JiimI, KafI, KhaaI,
LamI, MiimI, NounI, RaaI

Client 3 (AHCD Dataset)

SadI, ShiinI, SiinI, TaaI, ThaI,
ThaaI, WawI, YaaI, Zadi

5.3 Results

Figure 18 illustrates the set of character classes that were successfully recognized by the global model after federated training with missing classes at the client level. We observe that the detected classes mainly correspond to those present in local client datasets, while classes that are absent or under-represented at some clients are less consistently recognized. This confirms the negative impact of class-wise data fragmentation on the generalization capability of the global federated model.



Figure 18: Detected character classes after federated training with missing classes at clients

Global Performance. The global model achieved an overall accuracy of **84.98%** when evaluated on the complete test set. The total training time was approximately **317 seconds** (about **5.3 minutes**). Compared to centralized learning and full-class federated learning, this performance drop is expected due to the reduced and fragmented availability of class labels across clients.

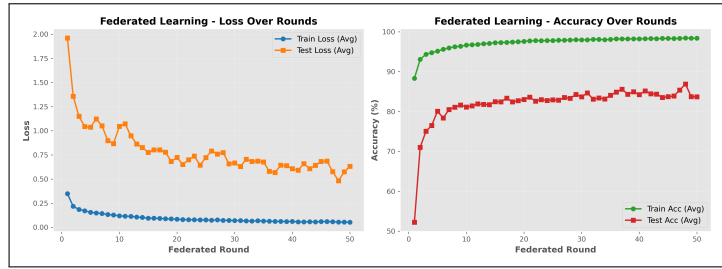


Figure 19: Training and validation metrics of the federated model with missing classes

5.3.1 Classes with Low Recognition Performance

Figure 20 presents a detailed per-class performance analysis. We observe that the classes with the lowest recognition rates correspond to letters with low frequency in the training data. This class imbalance negatively affects the model's ability to learn robust representations for these letters. To address this issue, we will extend the client datasets by adding more samples for the under-represented letters and analyze the impact on the global model performance.

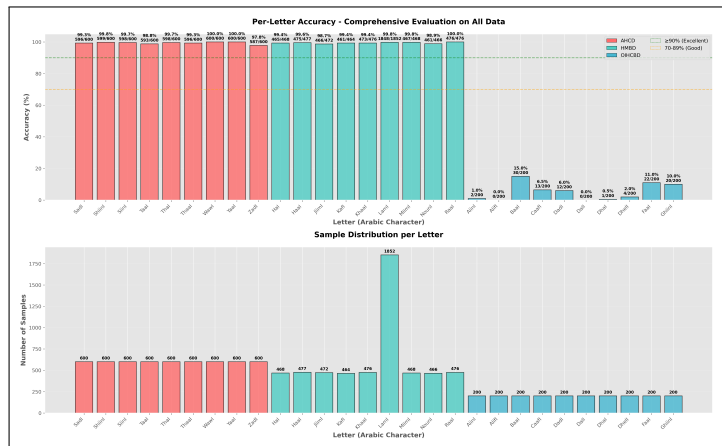


Figure 20: Per-letter performance analysis highlighting low-frequency and difficult classes

5.4 Flower Federated Learning with More Balanced Missing Classes

As shown in Figure 20, the OIHCBD dataset was the most affected by missing classes. To reduce this imbalance, we augmented OIHCBD by adding samples of the missing letters from the other datasets. This results in a more balanced class distribution across clients. We then retrained the federated model under these new constraints in order to evaluate the impact of class balancing on performance.

5.4.1 Results

Figure 21 shows the per-letter performance after balancing the class distribution. We observe that classes with higher frequency tend to dominate the average performance and bias the global model. To mitigate this effect, we enforce a fixed number of samples per class globally, ensuring that each letter contributes equally during training. This strategy helps reduce gradient bias toward dominant classes and leads to a more stable and fair learning process across all letters.

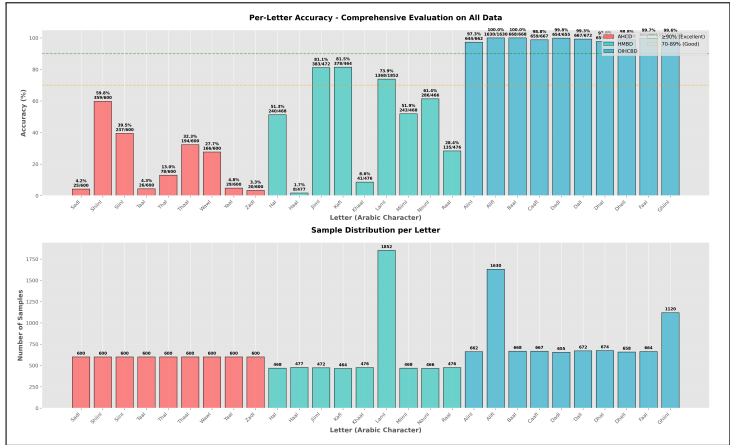


Figure 21: Per-letter performance after balancing class distribution across clients

5.5 Comparative Study: Centralized vs Federated Learning

Table 7 presents a comparison between centralized learning, standard federated learning, and federated learning with missing classes. All experiments use the same baseline CNN architecture to ensure a fair comparison.

Table 7: Comparison between Centralized Learning, Federated Learning, and Federated Learning with Missing Classes

Training Strategy	Number of Parameters	Training Time	Overall Accuracy
Centralized Learning (Baseline CNN)	1,157,085	170.11 s ($\approx$ 2.8 min)	96.50%
Federated Learning (Full Classes)	1,157,085	711.32 s ($\approx$ 11.9 min)	91.50%
Federated Learning (Missing Classes)	1,157,085	317.00 s ($\approx$ 5.3 min)	84.98%

6 Conclusion

From this TP, we draw several important lessons regarding both model design and federated learning:

- A more complex model does not necessarily guarantee better performance. Simplicity with proper training can often achieve comparable or even superior results.
- Federated learning enables the creation of a global model without sharing private data, preserving data privacy while still benefiting from distributed training.
- Careful and unbiased selection of data when updating model weights is crucial. Ensuring balanced class representation prevents gradient bias and improves the generalization of the global model.
- The concept of training rounds in federated learning is essential for coordinating local updates and effectively aggregating parameters across clients.

Our comparative analysis shows that:

- The centralized model achieves the highest performance due to access to all training data.
- Federated learning with complete class availability exhibits a moderate drop in accuracy and requires more training time due to communication overhead and decentralized optimization.
- Federated learning with missing classes suffers further performance degradation, demonstrating the significant impact of class-wise data fragmentation on the global model’s generalization ability.

Overall, these results highlight the trade-off between privacy preservation and model performance in federated learning, as well as the importance of careful data management and model design when working in distributed and non-IID settings.

References

- [1] A. El-Sawy, M. Loey, and H. EL-Bakry, *Arabic handwritten characters recognition using convolutional neural network*, WSEAS Transactions on Computer Research, vol. 5, pp. 11–19, 2017. https://doi.org/10.1007/978-3-319-48308-5_54
- [2] H. Balaha, *HMBD-v1: Handwritten Arabic Multi-position Dataset*, GitHub Repository, 2020. <https://github.com/HossamBalaha/HMBD-v1?tab=readme-ov-file>
- [3] C. Boufenar et al., *Off-line Isolated Handwritten Arabic Characters Database (OIHACDB-28)*, Unpublished dataset, 2020. <https://www.mediafire.com/file/diikxls8qdi3ibb/OIHACDB.rar/file>
- [4] Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press.
- [5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.
- [6] Y. LeCun, P. Haffner, L. Bottou, and Y. Bengio, *Object recognition with gradient-based learning*, Shape, Contour and Grouping in Computer Vision, Springer, 1999.
- [7] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, *Communication-efficient learning of deep networks from decentralized data*, Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.
- [8] P. Kairouz et al., *Advances and open problems in federated learning*, Foundations and Trends in Machine Learning, vol. 14, no. 1–2, pp. 1–210, 2021.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, *Deep residual learning for image recognition*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.